Technical design: Weibee Steel bid intelligence

Technical design: Weibee Steel bid intelligence

Version 0.1, October 2026. Companion to the plain-language README. This document is for the reader who wants to know how the numbers are produced, why the design choices were made, and what would change on production data.

1. Purpose and scope

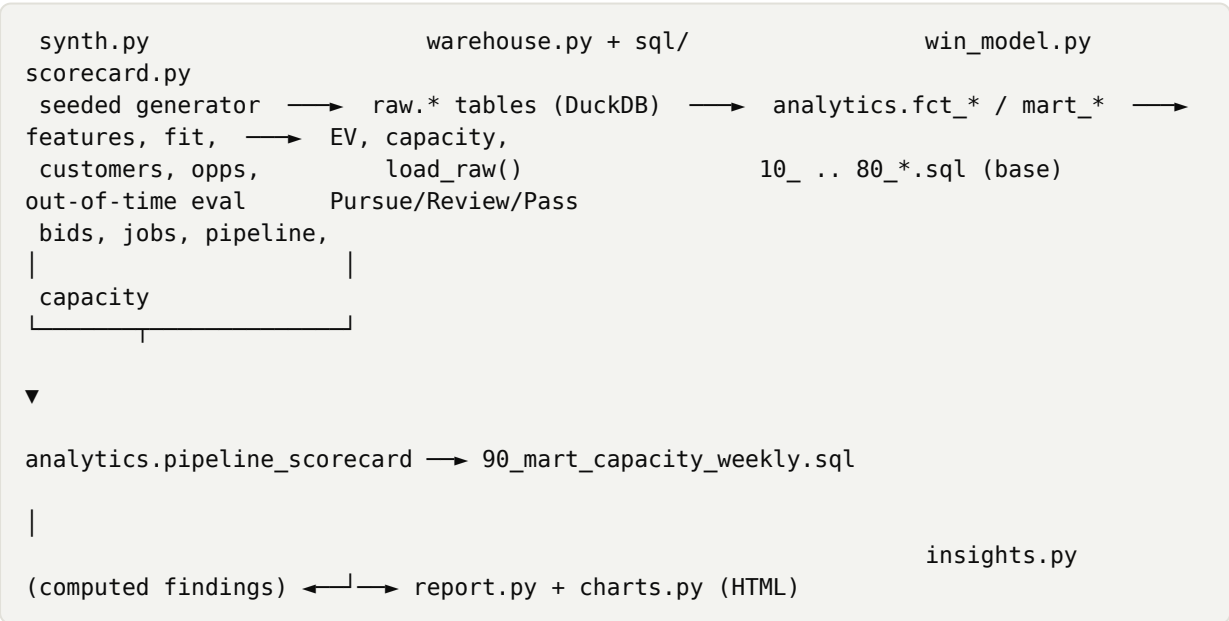
The system turns a structural-steel fabricator's bid history into three things:

1. **Descriptive marts**: hit rates, dollars won per estimating hour, pricing feedback, estimated versus realised margin, customer behaviour, shop load.
2. **A win-probability model** fitted to that history and evaluated out of time.
3. **A bid/no-bid scorecard** that ranks the forward pipeline by expected gross-margin dollars per estimating hour, checks shop capacity, and explains each call in words.

Everything runs locally from one command in under a second on a laptop. Inputs are five tables; the output is one HTML report, a set of CSV exports and a model card. No credentials, no services.

Out of scope: the synthetic data is a stand-in for a real bid log. Section 9 covers what changes when the real one arrives.

2. Architecture



Module responsibilities:

Module	Lines	Responsibility
`config.py`	120	Business parameters (capacity, rates, thresholds) and the project-type table that drives the generator
`synth.py`	330	Deterministic synthetic history with planted structure
`warehouse.py`	60	DuckDB connection, raw load, mart build in two stages
`sql/*.sql`	9 files	One mart per file; plain SQL; dollar-weighted ratios only
`win_model.py`	210	Feature construction, ridge logistic regression (numpy), AUC/Brier/calibration
`scorecard.py`	200	Fitted business inputs, EV, capacity check, recommendation rules
`insights.py`	190	Findings sentences computed from the marts
`charts.py`	300	Inline-SVG bar, dumbbell, histogram, line, stacked-area, calibration
`report.py`	450	Page assembly, tables, tiles, glossary, CSS with light and dark tokens
`__main__.py`	100	CLI: `python -m bid_intelligence run --seed N --out DIR --db FILE`

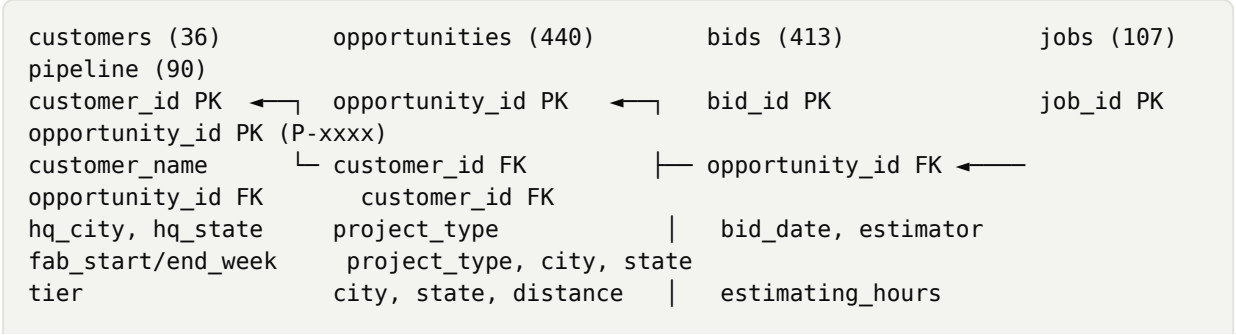
Dependencies: `duckdb`, `pandas`, `numpy`. Dev: `pytest`, `ruff`. Python 3.11+.

Why these choices

- **DuckDB** gives a real SQL warehouse in a file with zero setup, and the SQL ports to BigQuery, Snowflake or Postgres with small dialect changes. The marts are the contract; the engine is swappable.
- **Plain SQL files, one per mart**, so an estimator can read where a number comes from without reading Python. The build order is the filename prefix.
- **numpy logistic regression** instead of scikit-learn or a gradient-boosted model. With a few hundred bids an interpretable model with reported coefficients is worth more than a point of AUC, and it removes a dependency.
- **Inline SVG** instead of a charting library. The report is one 130 kB file that opens anywhere, prints to PDF cleanly, and every mark carries a native tooltip.

3. Data model

Five raw tables. The full field list with core/optional flags is in `data_dictionary.md`.



```
erection_end_week      distance_miles, est_tons
first_contact_date    est_tons, complexity    │   est_* cost build-up     status
complexity, delivery_method
                      delivery_method, scope  │   markup_pct, bid_price   actual_*
costs/hours   scope, bid_due_date
                      bid_due_date            │   num_bidders
change_orders_*        fab_start_week, fab_weeks
                      fab_start_week,         │   result, result_date
final_revenue          status (lead/invited/bidding)
                      fab_weeks               │   lost_reason
schedule_variance_days lead_source
                      decision, no_bid_reason │   competitor_low_price    ncr_count
                                              │
days_to_final_payment
capacity (weekly): week_start, capacity_tons  └─ one bid per bid opportunity; no_bid
opportunities have none
```

Grain rules that matter downstream:

- An **opportunity** exists whether or not we bid. Logging no-bids is what makes the bid/no-bid process learnable.
- A **bid** has exactly one `result` in {won, lost, cancelled, pending}. Only won and lost are "decided" and enter hit-rate denominators.
- A **job** exists only for won bids. Actuals are null until `status = complete`.
- **Pipeline** rows share the opportunity schema but have no bid; the scorecard fills in expected revenue from history.

4. Synthetic data generator

`synth.Generator(seed)` is fully deterministic from one `numpy.random.default_rng(seed)`. It exists to give the analytic layer true structure to recover; it is not a simulation of Weibee Steel.

Planted structure (all in `config.py` and `synth.py`, documented inline):

Mechanism	Where	Effect
Project-type fit	`ProjectType.win_logit`, `margin_slip`	Warehouse, industrial, K-12 win more and keep margin; healthcare, podium, hospitality the reverse
Customer relationship	hidden `relationship` in [0, 1] per GC	+1.7 log-odds across the range; some GCs award, some shop
Price sensitivity	hidden `price_sensitivity`	Lowers odds as markup rises; widens the price gap when we lose
Distance	`-0.0035 * max(0, miles - 80)`	Odds fall with distance; far erection overruns field hours
Bidder count	`-0.22 * max(0, n - 3)`; Poisson by delivery method	Hard bids have more bidders

Mechanism	Where	Effect
Size and complexity	`-0.55 * max(0, ln(tons/700))`, `-0.18 * max(0, cx - 3)`	Very large or complex work wins less and slips more
Change-order friction	hidden `co_friction`	Lowers CO approval rate, adds unpaid CO cost

The intercept of the hidden win model is solved by bisection so the mean true win probability is 29%, so hit rate does not drift with the parameter table. Results decided after the snapshot date are marked `pending`. The generator also returns `_truth` (true win probabilities per bid), used only by tests; it is never loaded into the warehouse.

Scale: 440 opportunities over 4.75 years, 413 bids, ~107 wins, 90 pipeline opportunities over the next 24 months, shop capacity 120 then 140 tons/week. Won tonnage is calibrated to run the shop at roughly 80% utilisation.

5. Warehouse marts

Built by `warehouse.build_marts(con, stage)`. Stage `base` runs `10_` to `80_`; stage `final` runs `90_` after the scorecard is written back, because the capacity mart needs win probabilities.

File	Grain	Key definitions
`10_fct_bids`	bid	`is_decided`, `is_won`, size and distance bands, `price_per_ton`, `est_gross_margin_pct = (bid_price - est_direct_cost) / bid_price`, `price_gap_pct = (bid_price - competitor_low_price) / bid_price` on price losses, `money_left_pct` on wins
`20_fct_jobs`	job	`actual_gross_margin_pct = (final_revenue - actual_direct_cost) / final_revenue`, `margin_slip_pts = est - actual`, shop and field hour variance, CO approval rate
`30_mart_bid_results_quarterly`	quarter	hit rate by count and by value, tons, estimating hours, won $ per estimating hour
`40_mart_hit_rate_by_dimension`	dimension × value	the same measures cut by project type, size band, distance band, delivery method, state, customer tier (a `union all` of six cuts)
`50_mart_customer_scorecard`	customer	bids, hit rate, won $, won $/est hour, lost on price, median price gap, hard-bid share, realised margin, CO approval, days to pay, `relationship_class`

File	Grain	Key definitions
`60_mart_estimate_vs_actual`	dimension × value	dollar-weighted estimated vs realised margin by project type, complexity, distance band, tier, plus an `all` row
`70_mart_pricing_feedback`	delivery method × gap bucket	histogram of price gaps
`80_mart_backlog_weekly`	week	won tonnage spread evenly over each job's fabrication window, against capacity
`90_mart_capacity_weekly`	week	backlog + Σ p_win × tons / fab_weeks over each pipeline window; `projected_utilisation`

Conventions enforced throughout:

- **Ratios are sums divided by sums**, never averages of ratios. Hit rate by value is Σ won $ / Σ bid $. Margins are revenue-weighted. Tests check this.
- **Gross margin is before overhead.** Overhead is tracked but kept out of the margin definition so estimated and realised figures are comparable.
- **Decided bids only** in every hit-rate denominator.
- **Relationship class** is a first cut: A if ≥ 4 decided bids, hit rate ≥ 40% and realised margin within 2 points of the shop average; B if ≥ 3 bids and hit rate ≥ 22%; "New / thin" under 3 bids; C otherwise. It is meant to be overridden by account knowledge.

6. Win-probability model

6.1 Features (`win_model.design_matrix`)

Feature	Form	Note
`log_tons`	ln(est_tons)	
`distance_100mi`	miles / 100	
`complexity`	1 to 5	
`markup_pct_x10`	markup × 10	known at bid time; set to the target markup when scoring pipeline
`num_bidders`	count	from the bid tab; delivery-method mean for pipeline
`gc_prior_hit_rate`	(prior wins + k·base) / (prior bids + k), k = 3	**computed as of each bid date**, so a bid never sees its own outcome
`log1p_gc_prior_bids`	ln(1 + prior bids)	relationship depth
`fab_and_erect`	0/1	
`method=*`	3 dummies	hard bid is the reference level

Feature	Form	Note
`type=*`	11 dummies	warehouse is the reference level

21 features, ~300 training rows. Features are standardised; the fitted coefficients are therefore "log-odds per standard deviation", which is what the model card plots.

6.2 Estimator

Ridge-penalised logistic regression, fitted by Newton's method on the standardised design matrix with an unpenalised intercept, λ = 1.0. Converges in under ten iterations. Implemented in ~25 lines of numpy (`fit_logistic`) so there is no dependency and nothing hidden.

6.3 Evaluation (`win_model.train`)

Out-of-time split: fit on decided bids with `bid_date < 2025-07-01` , test on the rest. Metrics: AUC (rank-based, tie-aware), Brier score against the base-rate Brier, log loss against base-rate log loss, and a five-bin calibration table. The model is then re-fitted on all decided bids for scoring, and the hold-out diagnostics are carried on the final model object and written to `model_card.json` .

Sample run (seed 5): test n = 100, base rate 29%, **AUC 0.75**, Brier 0.182 vs 0.206 baseline, log loss 0.537 vs 0.603. The largest standardised effects are bidder count (−0.62), distance (−0.50), markup (−0.50), complexity (−0.38) and negotiated delivery (+0.32). On the generator's own true probabilities the achievable AUC is about 0.71 to 0.75 depending on seed; bidding is noisy by construction, and the model is close to that ceiling.

Known limits: with 12 type dummies on 300 rows, individual type coefficients are noisy and the ridge penalty pulls them toward zero. The customer prior does most of the relationship work. A real deployment would re-fit quarterly and watch calibration drift.

7. Scorecard

7.1 Fitted business inputs (`scorecard.py`)

- **Estimating effort**: `ln(hours) = a + b·ln(tons) + c·complexity` , least squares on our own bid log. Sample fit: b = 0.64, c = 0.088, residual σ = 0.19 in log space.
- **Price per ton** by project type: median over decided bids.
- **Expected realised margin** by project type: realised gross margin from `mart_estimate_vs_actual` , shrunk toward the shop-wide figure with five pseudo-jobs so thin history does not swing it.
- **Yardstick**: gross-margin dollars won per estimating hour over the trailing two years, Σ (bid_price × est_margin × is_won) / Σ estimating_hours. Sample: about $2,500 per hour.

7.2 Expected value

For each pipeline opportunity i:

```
p_i        = model win probability (markup = target, bidders = method mean, customer
prior from full history)
revenue_i  = est_tons_i × price_per_ton[type_i]
margin_i   = expected realised margin[type_i]
hours_i    = exp(a + b ln tons_i + c cx_i)
EV_i       = p_i × revenue_i × margin_i − hours_i × estimator_rate
EV/hr_i    = EV_i / hours_i
```

Ranking is by EV per estimating hour because estimating hours are the binding constraint, not revenue. Thresholds are relative to the yardstick (Pursue ≥ 80%, Review ≥ 35%), so they re-scale with the shop's own economics instead of being hard-coded dollars.

7.3 Capacity check

Let `load(w)` be the weekly tons from `mart_backlog_weekly`, and `W_i` the set of weeks in opportunity i's fabrication window. Then

```
others(w)        = Σ_{j≠i} p_j × tons_j / fab_weeks_j   for w in W_j
util_if_won_i    = mean over w in W_i of ( load(w) + others(w) + tons_i / fab_weeks_i
) / capacity(w)
```

Above 105% is a hard flag (Pass); above 90% is a soft flag (Review). The job is counted in full, not at its odds, because the question is "what happens if we win it".

7.4 Recommendation rules (`scorecard.recommend`)

In order: hard flags (shop over capacity if won; win odds below 12%) → Pass. Non-positive EV → Pass. EV/hr ≥ 0.8 × yardstick with no soft flags → Pursue; with only "no bid history with this GC" → Pursue; with other soft flags (far erection, shop tight, low realised margin on the type) → Review. Between 0.35 and 0.8 × yardstick → Review. Below → Pass. One override: a Pass for a new regional or national GC on one of our three best-fit project types becomes Review, labelled strategic. Every flag and positive factor is concatenated into the `reasons` column.

The rules are plain `if` statements on purpose. They are the part an estimating manager should edit.

8. Report, insights and charts

`insights.generate(ctx)` returns a list of {title, text, action} dicts computed from the marts: best and worst project types with their margin slip and share of estimating hours; pricing-feedback coverage and median gap; price-shopper GCs versus A-class accounts; distance effect; backlog weeks, quarterly projected utilisation and the tonnage gap to 85% utilisation; overall estimate-vs-actual; model honesty. Each is a sentence template filled from numbers, so it changes with the data.

`report.render(ctx)` assembles one HTML file: header and navigation, a how-to-read box, the synthetic- data notice, six headline tiles, findings, then sections for hit rate, customers,

estimate vs actual, pricing feedback, capacity, the scorecard (summary plus full ranked table), the model card and a glossary. CSS uses design tokens with light and dark variants.

`charts.py` renders SVG strings with no JavaScript. Palette hues are fixed to entities (estimate blue, actual orange, backlog blue, pipeline amber, status colours reserved). Every mark has a `<title>` tooltip; charts have a legend when there is more than one series and never a dual axis.

9. From synthetic to production data

Component	Sample	Production
Source	`synth.generate()`	CSV or database extracts from the estimating log, job cost and the GC database, through `warehouse.load_raw()` with the same five table names
Snapshot date	fixed 2026-09-28	run date
Capacity	constant with one step	shop calendar including shifts, holidays and outsourced fabrication
Price per ton	type median	the actual estimate when one exists; type median only at lead stage
Model features	as above	add estimator, bid-tab bidder identities, days since last award from that GC, change-of-contact flags
Thresholds	relative to yardstick	same; review quarterly with estimating
Report	static HTML	same file on a shared drive, or the marts behind Power BI / Looker

What has to be true of the real intake for the marts to work: every invitation logged (bid or not), every bid with hours and a cost build-up, every result with a reason, and job-cost closes tied back to the bid. The low-bid number on losses is the field most shops do not collect; its coverage is reported as a KPI because the pricing-feedback section is only as good as it.

10. Testing and reproducibility

23 tests in `tests/`, run with `make test` in about two seconds:

- **Generator**: determinism across runs; key integrity across tables; economic sanity ($/ton, hit rate, margin, price-gap bounds); pending bids have no outcome; planted structure is recoverable (warehouse beats healthcare).
- **Warehouse**: `fct_bids` reconciles to raw; hit rates are bounded and every dimension cut partitions the same bids; quarterly sums match the fact table; slip equals estimate minus actual; the `all` row is dollar-weighted; backlog spreading conserves tonnage; every customer gets a class.

- **Model**: AUC against known values; the customer prior has no leakage (verified on a hand-built three-bid history); logistic fit recovers a planted signal; out-of-time AUC > 0.55 and Brier no worse than baseline on generated data.
- **Scorecard**: one row per pipeline opportunity, ranks are a permutation, EV arithmetic matches the formula, low odds always Pass, estimating-hours fit is increasing in tons, the final capacity mart builds after scoring.
- **Report**: chart helpers return SVG; the end-to-end CLI writes a report with every section anchor.

Reproducibility: `python -m bid_intelligence run --seed N` regenerates everything from one integer. The committed `output/` is seed 5. `ruff` enforces formatting and lint (line length 110).

11. Performance

Whole pipeline, laptop class CPU: generation 0.2 s, marts 0.1 s, model 0.1 s, scorecard 0.2 s, report 0.1 s. The scorecard's capacity loop is O(pipeline × weeks) in pandas and would stay under a second to several thousand opportunities; beyond that, move it into SQL alongside `90_mart_capacity_weekly`.

12. Known limitations

- Synthetic data: findings illustrate the kind of result, not Weibee Steel's business.
- Estimating hours are the only scarce resource modelled. Bonding, cash flow, crew availability and GC schedule risk belong in the reasons column once data exists.
- Expected revenue for pipeline work is a type median; real estimates will be better and the scorecard should prefer them when present.
- The win model does not model competitor identity or market cycle; both are worth adding when bid tabs are logged.
- The customer prior uses the full history when scoring pipeline work, which is correct for forecasting but means scorecard odds are not directly comparable with the back-tested ones.

13. Repository map

```
bid_intelligence/   package (see section 2)
bid_intelligence/sql/   10_fct_bids.sql .. 90_mart_capacity_weekly.sql
tests/              23 tests
docs/               README companions: HOW_IT_WORKS.md, TECHNICAL_DESIGN.md (this
file),
                    data_dictionary.md, memo.md, DEMO_GUIDE.md, report PDF, executive
deck, images/
output/             rendered report, CSV exports, model_card.json, findings.md,
```

```
data/*.csv
Makefile, pyproject.toml, requirements.txt
```